

Week 6 Assignment: Linear Q-Function Approximation

Feature design, semi-gradient updates, and stability evidence

CMPSC 480 - Student Question Deck

Assessment overview

Replace a Q-table with a fixed-size linear approximator and verify every update.

Coverage: Week 6: function approximation, semi-gradients, and the deadly triad

Points

10 total

Expected time

6-8 hours

Submission

Repository plus experiment memo

Learning objectives

- ▶ Implement a fixed-length state-action feature map.
- ▶ Compute Q values as vectorized dot products.
- ▶ Apply terminal-aware semi-gradient Q-learning.
- ▶ Demonstrate the one-hot tabular correspondence.
- ▶ Monitor numerical stability across controlled runs.

Requirements checklist

- ▶ Use Python 3.11 and NumPy; do not use a library that implements the assessed algorithm.
- ▶ Keep data, model, optimization, and evaluation responsibilities behind explicit interfaces.
- ▶ Validate shapes, ranges, finite values, empty inputs, and terminal or degenerate cases.
- ▶ Use deterministic seeds and record every configuration used for reported evidence.
- ▶ Include focused tests and a README with exact reproduction commands.

Task 1 - Features and dimensions (2 points)

Design a feature map whose length is independent of the number of raw states.

- a. Define at least four state-action features and justify their scales. [1 points]
- b. Prove by an executable assertion that parameter size does not grow when the environment map grows. [1 points]

Task 2 - Linear values and policy (2 points)

Implement Q -hat and epsilon-greedy action selection over legal actions.

- a. Compute $Q\text{-hat}(s,a;w)$ and test a known vector by hand. [1 points]
- b. Specify training and evaluation tie behavior. [1 points]

Task 3 - Semi-gradient update (4 points)

Implement the squared-TD-error semi-gradient rule with correct terminal masking.

- a. Derive the update from one-half times squared TD error. [2 points]
- b. Compute an update for $w=[0,0]$, $f=[1,2]$, reward=1, gamma=0.5, max-next-Q=4, and alpha=0.1. [1 points]
- c. State and test the terminal target. [1 points]

Task 4 - Experiment and stability (2 points)

Compare feature sets and report evidence beyond episodic return.

- a. Run one feature ablation and compare with a tabular small-state oracle. [1 points]
- b. Define an instability gate using at least three logged quantities. [1 points]

Edge cases and defensive programming

- ▶ Zero feature vector.
- ▶ Terminal next state.
- ▶ Feature dimension mismatch.
- ▶ All legal actions tie.
- ▶ Weight or TD error becomes nonfinite.

Submission instructions

- ▶ Submit source, tests, README, configuration, and the requested report.
- ▶ Provide one command that reproduces the primary result from a clean environment.
- ▶ Exclude credentials, generated caches, and unlicensed data.

Do not submit credentials, generated caches, or unapproved libraries.

Grading rubric

Category	Points	Required evidence
Features and dimensions	2	Feature implementation, dimensionality assertion, and feature table.
Linear values and policy	2	Vectorized <code>q_value</code> and deterministic evaluation tests.
Semi-gradient update	4	Update method, derivation, and hand-computed tests.
Experiment and stability	2	Ablation table, stability plots or summaries, and defensive tests.

Reflection prompt

In 150-250 words:

Identify one feature that generalized useful information and one feature that created coupling risk. Cite an ablation or trace.